CSC3060 Data Lab Report

Course: CSC3060 **Lab Title:** Data Lab — Bit-Level Arithmetic Implementation
Student: (Fill in Name & ID)
Date: (Fill in Submission Date)

1. Objective

The goal of this lab is to implement integer arithmetic operations using **bitwise operations only**, without relying on standard arithmetic operators such as +, -, *, /, and %. This exercise deepens understanding of **binary representation**, **two's complement arithmetic**, and **low-level CPU computation**.

The implemented functions include: - Addition - Subtraction - Multiplication - Division - Modulo

All implementations were validated using **GoogleTest**, and all test cases passed successfully.

2. Constraints and Rules

The following constraints were enforced in this lab:

- No direct use of arithmetic operators: +, -, *, /, %
- Use only bitwise operations: &, |, ^, ~, <<, >>
- Correct handling of:
    - Negative numbers
    - Overflow behavior
    - Edge cases (e.g., INT_MIN, INT_MAX, zero division scenarios handled safely)

These constraints simulate **low-level ALU (Arithmetic Logic Unit) behavior**.

3. Key Concepts Used

3.1 Two's Complement Representation

Integers are stored in two's complement form, which allows subtraction to be implemented as:

```
a - b = a + (~b + 1)
```

3.2 Addition Using Bitwise Operations

Addition was implemented using iterative carry propagation:

- XOR (^) computes sum without carry
- AND (&) computes carry bits
- Left shift (<<) propagates carry

This repeats until no carry remains.

4. Function Implementations and Logic

4.1 Addition

Idea: Simulate binary addition using XOR and carry propagation.

Steps: 1. Compute partial sum using XOR 2. Compute carry using AND 3. Shift carry left and repeat

Complexity: O(1) for fixed-width integers

4.2 Subtraction

Idea: Convert subtraction into addition using two's complement.

```
a - b = a + (~b + 1)
```

This allows reuse of the addition function.

4.3 Multiplication

Idea: Use shift-and-add method (binary long multiplication).

Steps: - If the lowest bit of multiplier is 1, add shifted multiplicand - Shift multiplicand left - Shift multiplier right

Handles sign by: - Recording sign - Converting operands to positive - Applying sign to result

4.4 Division

Idea: Implement binary long division without using `/`.

Steps: 1. Convert operands to positive 2. Repeatedly subtract largest shifted divisor 3. Accumulate quotient using bit shifts 4. Restore correct sign

This ensures accurate integer truncation toward zero.

4.5 Modulo

Idea: Compute remainder using the division logic:

```
remainder = dividend - (quotient × divisor)
```

This ensures modulo behavior consistent with C++ integer arithmetic.

5. Edge Case Handling

Case	Handling Strategy
Zero dividend	Return zero
Divide by ±1	Fast path optimization
INT_MIN overflow	Special handling to avoid UB
Negative numbers	Normalize sign and restore
Large random values	Verified using unit tests

6. Testing and Validation

All functions were tested using **GoogleTest**.

Test Coverage Includes:

- Positive values
- Negative values
- Mixed signs
- Zero cases
- Large numbers
- Powers of two
- Random values
- Overflow and underflow

Result Summary:

```
[==========] 60 tests from 5 test suites ran
[  PASSED  ] 60 tests
```

All tests passed successfully.

7. Performance Considerations

Since integers are fixed-width (32-bit), all operations run in **constant time**.

The algorithms mimic **hardware-level arithmetic**, making them efficient and scalable.

8. Lessons Learned

Through this lab, I gained deeper understanding of:

- How CPUs perform arithmetic internally
- Two's complement arithmetic and overflow behavior
- Bit-level manipulation techniques
- Writing safe, low-level arithmetic without built-in operators

This lab strengthened my ability to reason about **low-level system behavior** and **binary computation**.

9. Conclusion

This project successfully implemented all required arithmetic operations using only bitwise logic. All GoogleTest cases passed, confirming correctness and robustness.

The lab provided valuable insight into low-level arithmetic and CPU design principles.

10. Appendix

Screenshots of my dialogue with LLM

🧩 Step 1 — 先实现 `add(a, b)`（禁止控制流）

✅ **允许：位运算**

❌ **不允许：** `+`、`if`、`while`、`for`

💡 **原理（你报告可以写）**

二进制加法规则：

- **不进位和** = `a XOR b`
- **进位** = `(a AND b) << 1`
- 重复直到没有进位

🚀 Step 2 — `subtract(a, b)`（用 add + two's complement）

原理：

nginx 复制代码

```nginx
a - b = a + (~b + 1)
```

✅ 合规实现

cpp 复制代码

```cpp
int32_t subtract(int32_t a, int32_t b) {
    int32_t neg_b = add(~b, 1);
    return add(a, neg_b);
}
```

> ⚠️ 目标：
> - “不用 `/`”
> - “支持负数”
> - “向 0 截断（符合 C++）”
> - “用位移模拟“长除法””

🧠 核心思想（你报告可用）

二进制除法 = **从高位到低位尝试减去除数**

思路：

1. 记录结果符号
2. 转成正数
3. 从 bit 31 → bit 0
4. 如果 `(divisor << i)` ≤ remainder，则减去并设置商位
5. 恢复符号

实现：

```
a % b
```

并满足：

- 不用 `%`
- 符号规则和 C++ 一致（**余数符号跟 dividend = a**）
- 复用我们已经写好的：`divide()`、`multiply()`、`subtract()`

🧠 数学关系（报告可用）

```matlab
a % b = a - (a / b) * b
```

我们已有：

- `divide(a, b)` ✅
- `multiply(x, y)` ✅
- `subtract(x, y)` ✅